

Clarifying the Hypothesis: Entangled Enhancement in a Pipeline of Three Learned Models

Ifty Rashedul Arefin

Meeting 6 Report

September 23, 2026

Abstract

At the last meeting, Prof. Machida set three tasks. The first is to state the hypothesis clearly before running more experiments. The second is to reduce the amount of data so that the experiments fit the available computers. The third is to replace the prediction module and the planning module, which are fixed rules in the current study, with machine learning models. This report covers all three. It explains what the earlier work has shown and what our current study cannot show, and it states the hypothesis we will test next. It then describes the smaller dataset we have built and the learned models we propose for the two modules.

1 Where the Study Stands

The pipeline: Our study uses a *pipeline*, which is three modules connected in a line, where each module uses the output of the module before it. The *perception module* C_1 looks at a camera image and finds the *objects* around our car, such as other cars and pedestrians. The *prediction module* C_2 takes these objects and predicts where each object will move. The *planning module* C_3 takes these predictions and chooses the path that our car will drive over the next 3 seconds, which we call the *planned path*. A module that comes earlier in the line is *upstream* of a module that comes later, and the later module is *downstream*. All data come from nuScenes, a public driving dataset recorded in Boston and Singapore. The dataset is divided into *scenes*, which are driving clips of about 20 seconds, and the correct position of every object is labelled by hand. We call these labelled values the *ground truth*.

The theory: Our work builds on two papers by Wang and Machida [1,2], which we call the *base papers*. They define *entangled enhancement* as a retraining that improves one model but fails to improve the whole system, because the models depend on each other. Their mathematical models of when to retrain need one number that has never been measured. This number, called c_1 , is the probability that retraining the upstream model leaves the downstream model working. They set $c_1 = 0.75$ by assumption, which means that three out of four retrainings are safe. In their reply to the reviewers they explain that the number could not be measured, because no records of real systems under retraining exist.

The paper: Our paper retrained C_1 on Singapore images after it had been trained on Boston images. We call each such retraining an *update*. We measured the *planning error*, which is the distance between the planned path and the path the human driver actually drove, 3 seconds ahead. In every update C_1 became better at finding objects. In 6 of 15 updates the planning error became larger at the same time. We then repeated the updates with training data in which

images of rare object types were repeated more often, and the planning error became larger in 4 of a further 12 updates. We are not aware of an earlier count of this kind, and it is the evidence from a real system that the reviewers of the base papers asked for.

The gap: In our pipeline only the perception module C_1 (YOLOv11n) is a machine learning model that learns from data. The prediction module C_2 and the planning module C_3 are fixed formulas. The prediction module assumes that every object keeps moving at the same speed, and the planning module follows hand-written rules, such as slowing down when an object is close. Because they never learned anything, they cannot have become used to the mistakes of the old C_1 . The theory of entangled enhancement says that the damage happens because a downstream model has learned to work around the old mistakes of the upstream model, so it is disturbed when those mistakes are fixed. In our experiment the planning error became worse for a different and simpler reason. The retrained C_1 found objects more accurately, so a rule in the planning module that reacts to nearby objects was triggered more often, and the planned path moved away from the path of the human driver. The experiment log also shows that this happened only after we tuned that rule to react to distant objects. The result is therefore real, but it does not test what the theory claims.

From Meeting 1 our plan was to use a learned model in all three modules. We changed C_2 and C_3 to fixed rules in Meeting 4 only because we did not have enough computing power. In the next stage we return to learned models for C_2 and C_3 , so that we can test whether they really learn the old mistakes of C_1 and are harmed when C_1 improves.

2 The Hypothesis

H1: In a pipeline in which all three modules are learned models, retraining only the perception module on a new city will make the perception module better and, at the same time, make the planning error larger. The reason is that the prediction module and the planning module were trained on the output of the old perception module and have learned to work around its mistakes.

We write δ_1 for how much C_1 improves after the update, measured by how accurately it finds objects, and Δ_3 for how much the planning error grows after the update. H1 then says

$$\delta_1 > 0 \quad \text{and} \quad \Delta_3 > 0 \quad \text{when } C_2 \text{ and } C_3 \text{ were trained on the output of the old } C_1. \quad (1)$$

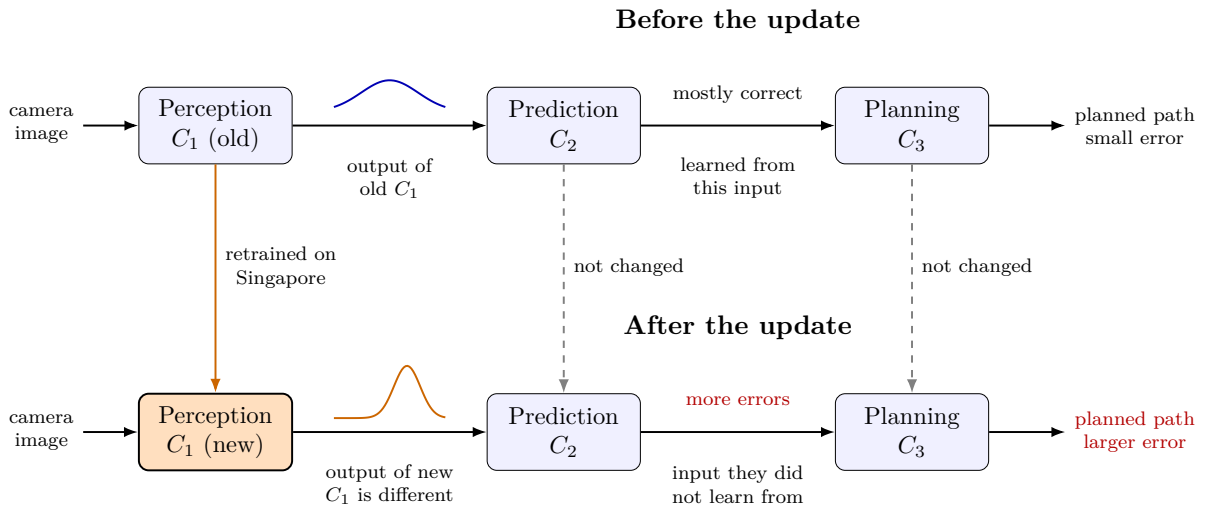


Figure 1: Hypothesis H1 in one picture

Figure 1 shows this idea. Only the perception module C_1 is retrained. It becomes better at finding objects, but the kind of output it produces changes, shown by the different curve. The prediction module C_2 and the planning module C_3 are not changed and were trained on the output of the old C_1 , so they now receive an input they did not learn from. They make more errors, and the planning error grows.

H1 names a cause, so we need a pipeline in which that cause is absent to compare against. To build one, it helps to separate training from driving. When the car drives, the modules are always connected in the pipeline, and the output of C_1 goes to C_2 and the output of C_2 goes to C_3 . Training happens earlier, and each module is trained on its own. During training we choose what input each module sees. The input of C_2 can be the ground truth, or it can be the output of C_1 with its mistakes. In our paper, feeding a module the ground truth instead of the output of the module before it was called *isolated mode*, and it was used only for testing. Here we use the same idea for training.

We therefore train C_2 and C_3 in three different ways, called *isolated training*, *old-pipeline training* and *new-pipeline training*, shown in Table 1. After training, the modules are connected into the normal pipeline in all three cases, and the same update of C_1 , from old to new, is applied.

Table 1: The three ways of training C_2 and C_3 . Only the input seen during training differs. When the car drives, the modules are connected in the same pipeline in all three cases.

Training	C_2 and C_3 learn from	What H1 predicts after the update
Isolated	the ground truth	a better C_1 helps or changes nothing
Old-pipeline	the output of the old C_1	the planning error grows, and grows more than with isolated training
New-pipeline	the output of the new C_1	the growth seen with old-pipeline training disappears

With isolated training, C_2 and C_3 learn from the ground truth, so they have never seen the mistakes of any C_1 . When C_1 improves, its output comes closer to the ground truth that they learned from, so the planning error should become smaller or stay the same. With old-pipeline training, C_2 and C_3 learn from the output of the old C_1 , so they also learn its mistakes. This is the case that H1 is about, and the planning error should grow, and grow more than with isolated training. With new-pipeline training, C_2 and C_3 are trained again on the output of the new C_1 , so they learn the new C_1 instead of the old one. If the growth with old-pipeline training was caused by the old mistakes, it should disappear with new-pipeline training.

H1 is therefore supported if the planning error grows more with old-pipeline training than with isolated training, and does not grow with new-pipeline training. H1 is rejected if isolated and old-pipeline training give the same growth, because then learning the old mistakes made no difference. H1 is also rejected if the planning error grows with none of the three.

3 Two Possible Causes

The base papers show that an upstream improvement can harm the system in two different ways. In their two examples [2], the downstream model *was* retrained on the new upstream output and still became worse. The upstream model had improved on cases that the downstream model did not need, and had become worse on the cases it did need.

- **Cause 1 (learned mistakes):** The downstream modules learned to work around the old mistakes of the upstream module and are disturbed when those mistakes change. This is

the cause described in our paper.

- **Cause 2 (uneven improvement):** The upstream module improves on average but becomes worse on the cases that matter downstream, for example distant objects or rare object types. This is the cause in the examples of the base papers.

The three ways of training tell the two causes apart. If the planning error still grows with new-pipeline training, where C_2 and C_3 were retrained on the new C_1 , then Cause 2 is present. If it grows only with old-pipeline training, then Cause 1 explains it. Both causes can also act at the same time.

H1a (learned mistakes): The planning error grows more with old-pipeline training than with isolated training, and new-pipeline training brings it back to the level of isolated training.

H1b (uneven improvement): The planning error still grows with new-pipeline training, even though every module was retrained on the new output. Splitting δ_1 by object type and by distance will then show where C_1 became worse.

4 The Second Hypothesis: How Often It Happens

H2: Entangled enhancement is not a rare event. We will run a set of updates that differ in the amount of training data, the number of training rounds and the *random seed*, which is the starting number for the random choices made during training. With old-pipeline training, the fraction of these updates that meet condition (1) will be clearly above zero, and it will be higher than with isolated training.

This fraction is a measured value of $1 - c_1$, the number that the base papers had to assume. The base papers recommend a different retraining rule when $c_1 < 0.1$, so a value measured on a real system can be used there directly. Our present counts, 6 of 15 and 4 of 12, are values of the same kind, but they come from a pipeline that cannot show Cause 1.

5 Checks That Stay in Place

- **Modules that were not retrained do not change:** We measure each downstream module in two ways. In the first way it receives the ground truth as input, and in the second way it receives the output of the module before it. Because C_2 and C_3 are not changed by the update, their result with the ground truth as input must stay exactly the same, digit by digit. If it changes, the measurement is faulty.
- **Random variation:** A growth of the planning error counts only if it is larger than the growth caused by retraining the *old* C_1 with nothing changed except the random seed. This also decides whether the smallest growth in our paper, 0.008 m, is real.
- **Two planning measures:** We report the planning error together with the *collision rate*, which is the fraction of scenes in which the planned path hits an object. Our paper showed that the two measures can move in opposite directions.
- **The planning module must use the objects:** Before any of the three ways of training is run, we remove the objects from the input of the learned C_3 and check that the planned path changes. A planning module that ignores the objects cannot be affected by C_1 at all, and Zhai et al. [3] show that such planning modules still score well on nuScenes.

6 What Would Make the Hypothesis Fail

1. Isolated training and old-pipeline training give the same growth of the planning error. Then learned mistakes are not the cause.

2. The planning error grows with none of the three ways of training. Then the effect in our paper came only from the fixed rules of the planning module.
3. The learned planning module ignores the objects. Then the experiment cannot test H1 and must be redesigned before any updates are run.

7 Why This Hypothesis

H1 asks the question of Meeting 1 in a form that an experiment can answer. It keeps the definitions, measures and checks of our paper. It tests the cause that the base papers assume, on the kind of real system that their reviewers asked for, and H2 gives a measured value for c_1 . It also explains why the effect in our paper appeared only after the planning module was tuned, because a planning module made of fixed rules cannot learn the mistakes of C_1 .

8 What Follows From It

8.1 Task 2: Reducing the Data

At the last meeting, Prof. Machida pointed out that the dataset was too large for the available computers and asked us to reduce the number of samples. We have therefore built a smaller dataset of 350 scenes. Table 2 compares it with the dataset used in the paper. The paper loaded all 850 scenes of nuScenes, a 45 GB dataset, and used 559 of them. The new dataset keeps only the 350 scenes we need, with the full-size images of the front camera and the label files. Its size is 4.6 GB, about ten times smaller. Boston is split into two separate groups of scenes, one to train C_1 and one to train C_2 and C_3 , because a perception module is much more accurate on the scenes it was trained on [4], and C_2 and C_3 must learn from the realistic mistakes of C_1 . The 50 test scenes are the same as in the paper, so the new results can be compared with the old ones. A small first run will retrain the old C_1 with two random seeds to show whether 350 scenes are enough. If they are not, we will add scenes.

Table 2: The dataset used in the paper and the reduced dataset. Every scene gives about 40 front-camera images. No image is missing in the reduced dataset.

Used for	Paper (previous)	Reduced (current)
Training the old C_1 (Boston)	280 scenes	100 scenes, 4,008 images
Training C_2 and C_3 (Boston)	none, fixed rules	100 scenes, 4,046 images
Updating C_1 (Singapore)	229 scenes	100 scenes, 3,997 images
Testing (Singapore)	50 scenes	the same 50 scenes, 2,020 images
Scenes used	559	350
Size of the dataset	45 GB	4.6 GB

8.2 Task 3: Learned Models for C_2 and C_3

Table 3 lists two published models for each module, a first choice and a second choice in case the first fails. All four are machine learning models with public code. For C_2 , the first choice is AutoBot-Ego [5], a small neural network with about 1.5 million learned numbers that was trained on all of nuScenes in about 3 hours on one graphics card. It takes the positions of the objects found in the last few images and predicts where each object will move, and it can learn by itself which found objects are the same object in different images [6]. If it does not learn well or trains too slowly, the second choice is Social-LSTM [7], an older and simpler network that reads

the past positions of each object one step at a time. It needs the same object to be matched across images first, so in that case a second small neural network learns this matching from the object identities labelled in nuScenes. For C_3 , the first choice is PlanT [8], a small neural network that takes a list of objects, here with the positions predicted by C_2 , and outputs the planned path. If it does not learn well, the second choice is AD-MLP [3], the simplest published planning model for nuScenes, to which we add the predicted positions of the nearest objects as input. For both choices of C_3 we do not give the speed of our car as input, because Zhai et al. [3] showed that a planning module with this input can ignore the objects. We also move to the second choice for C_3 if the check in Section 5 shows that it ignores the objects. Every experiment starts with the first choices, and a module is changed only when it fails.

Table 3: Proposed learned models for C_2 and C_3 .

Module	First choice	Second choice	When we switch
Prediction C_2	AutoBot-Ego [5]	Social-LSTM [7], with a learned matching network	it does not learn well or trains too slowly
Planning C_3	PlanT [8]	AD-MLP [3], with object positions added	it does not learn well or ignores the objects

References

- [1] Z. Wang and F. Machida, “Maintaining performance of a machine learning system against imperfect retraining,” in *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 2024, pp. 1782–1787.
- [2] Z. Wang and F. Machida, “Exploiting the availability-continuity trade-off in imperfect retraining of machine learning systems,” in *2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*, 2025, pp. 406–417.
- [3] J.-T. Zhai, Z. Feng, J. Du, Y. Mao, J.-J. Liu, Z. Tan, Y. Zhang, X. Ye, and J. Wang, “Rethinking the open-loop evaluation of end-to-end autonomous driving in nusenes,” *arXiv preprint arXiv:2305.10430*, 2023.
- [4] Y. Xu, L. Chambon, É. Zablocki, M. Chen, A. Alahi, M. Cord, and P. Pérez, “Towards motion forecasting with real-world perception inputs: Are end-to-end approaches competitive?” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 18 428–18 435.
- [5] R. Girgis, F. Golemo, F. Codevilla, M. Weiss, J. A. D’Souza, S. E. Kahou, F. Heide, and C. Pal, “Latent variable sequential set transformers for joint multi-agent motion prediction,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [6] X. Weng, B. Ivanovic, K. Kitani, and M. Pavone, “Whose track is it anyway? Improving robustness to tracking errors with affinity-based trajectory prediction,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [7] A. Alahi, K. Goel, V. Ramanathan, A. Robicquet, L. Fei-Fei, and S. Savarese, “Social LSTM: Human trajectory prediction in crowded spaces,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 961–971.
- [8] K. Renz, K. Chitta, O.-B. Mercea, A. S. Koepke, Z. Akata, and A. Geiger, “PlanT: Explainable planning transformers via object-level representations,” in *Conference on Robot Learning (CoRL)*, ser. PMLR, vol. 205, 2022.